

Setor de Educação Profissional e Tecnológica Tecnologia em Análise e Desenvolvimento de Sistemas

DS141 - Estruturas de Dados II

07 - Árvores

Prof. Alex Kutzke

Junho de 2021

Slides adaptados do material produzido
pelo Prof. Rodrigo Minetto

Roteiro

Motivação e conceito

Árvores binárias

Operações básicas

Exercícios propostos

Motivação

Examinamos até agora estruturas de dados que podem ser chamadas de lineares, como vetores e listas. A importância dessas estruturas é inegável, mas:

- Elas não são adequadas para representar dados que devem ser dispostos de maneira **hierárquica**;
- É necessário buscar estruturas **mais eficientes** para a implementação de tabelas de símbolos (inserção, atualização, busca e remoção).

Exemplo de Representação Hierárquica

Sistema de arquivos: arquivos e diretórios dentro de diretórios, ...;



Árvores

- No contexto da computação são **estruturas de dados** não sequenciais que seguem o conceito de **hierarquia**.
- A forma mais natural de definir uma estrutura de árvore é usando a **recursividade**.

Definição

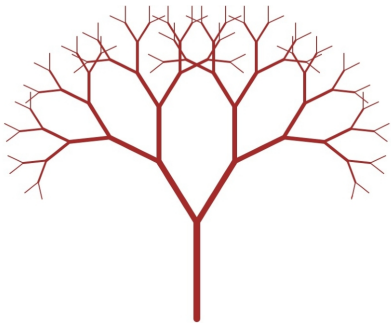
Uma árvore é um conjunto finito de n estruturas elementares chamados nós. Tal que se $n > 0$, então:

- existe um nó especial r , denominado raiz da árvore;
- os nós restantes são particionados em m conjuntos disjuntos (chamado de subárvores de r), tal que, cada um desses conjuntos é também uma árvore.

Árvores

Na computação:

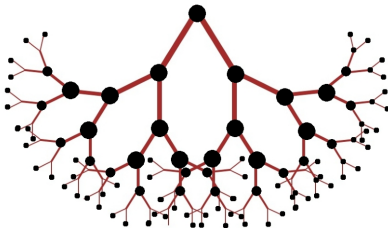
- as árvores são representadas de baixo para cima;



Árvores

Na computação:

- as árvores são representadas de baixo para cima;
- da raiz às folhas;
- cada bifurcação é um nó interno. Cada folha é um nó externo.



Conceitos

Grau de um nó número de subárvores de um nó;

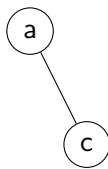
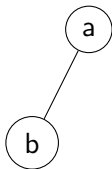
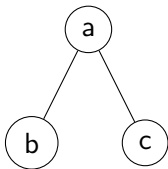
Grau da árvore grau máximo entre todos os nós;

Nó folha nó com grau 0 (sem subárvores).

- O nó raiz tem um **nível** 0, filhos do nó raiz tem nível 1 e, assim, sucessivamente;
- **Árvores binárias** são árvores cujos nós tem grau 0, 1 ou 2 (ou seja, tem no máximo 2 filhos);
- Um conjunto de árvores disjuntas forma uma floresta;
- Nós com filhos são chamados de **nós internos**;
- Nós sem filhos são chamados de **nós externos**.

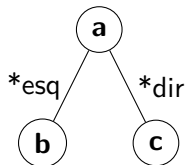
Árvore binária

- Uma subárvore de uma árvore binária é sempre especificada como sendo a subárvore da **esquerda** ou da **direita**;
- Qualquer uma das subárvores pode ser vazia;



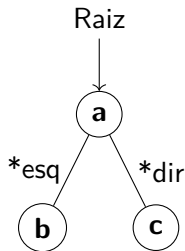
Estrutura

```
1  typedef struct arvore {
2      char info;
3      struct arvore *esq;
4      struct arvore *dir;
5  } Arvore;
```



Estrutura

```
1  typedef struct arvore {  
2      char info;  
3      struct arvore *esq;  
4      struct arvore *dir;  
5  } Arvore;
```



- Podemos representar a árvore com apenas um ponteiro para seu elemento raiz;

Operações

- Como todo **tipo abstrato de dados (TAD)**, a estrutura de dado deve vir acompanhada de **operações**;
- As operações dependem do uso que se pretende do TAD;
- Um conjunto de operações possíveis para um TAD de árvore binária poderia ser:

```
1 // arquivo arvore.h
2 Arvore* cria_arv_vazia (void);
3 Arvore* arv_constroi (char c, Arvore* e, Arvore* d);
4 Arvore* arv_libera (Arvore* a);
5 int     verifica_arv_vazia (Arvore* a);
6 int     arv_pertence (Arvore* a, char c);
7 void    arv_imprime (Arvore* a);
```

Operações - Criar árvore vazia

- Como a árvore é representada pelo endereço do nó raiz, uma árvore **vazia** pode ser representada por um ponteiro nulo:

```
1 Arvore* cria_arv_vazia (void) {  
2     return NULL;  
3 }
```

Operações - Construir árvore

- A construção de árvores não vazias pode ser realizada com 3 itens:
 1. Item a ser armazenado no nó raiz;
 2. Ponteiro para subárvore da esquerda;
 3. Ponteiro para subárvore da direita;

```
1  Arvore* arv_constroi (char c, Arvore* e, Arvore* d) {  
2      Arvore* a = (Arvore*)malloc(sizeof(Arvore));  
3      a->info = c;  
4      a->esq = e;  
5      a->dir = d;  
6      return a;  
7  }
```

Operações - Construir árvore

- A construção de árvores não vazias pode ser realizada com 3 itens:
 1. Item a ser armazenado no nó raiz;
 2. Ponteiro para subárvore da esquerda;
 3. Ponteiro para subárvore da direita;

```
1  Arvore* arv_constroi (char c, Arvore* e, Arvore* d) {  
2      Arvore* a = (Arvore*)malloc(sizeof(Arvore));  
3      a->info = c;  
4      a->esq = e;  
5      a->dir = d;  
6      return a;  
7  }
```

É claro que uma função de inserir elemento na árvore, ao invés de criar uma árvore para um único elemento, é muito mais útil. Mas veremos isso em breve.

Operações - Verifica árvore vazia

- Para verificar se uma árvore está vazia, basta um único teste:

```
1  int verifica_arv_vazia (Arvore* a) {  
2      return (a == NULL);  
3  }
```

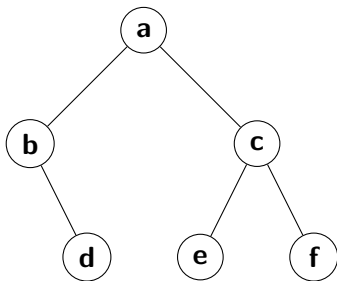
Operações - Liberar

- Para liberar o espaço alocado por uma árvore, precisamos liberar as subárvores também;
- Uma função recursiva pode resolver esse problema:

```
1  Arvore* arv_libera (Arvore* a) {  
2      if (!verifica_arv_vazia(a)) {  
3          arv_libera (a->esq);  
4          arv_libera (a->dir);  
5          free(a);  
6      }  
7      return NULL;  
8  }
```

- A ordem das chamadas recursivas e do free importa!

Exemplo: Construção de árvore



Exemplo: Construção de árvore

```
1  int main (int argc, char *argv[]) {  
2      Arvore *a, *a1, *a2, *a3, *a4, *a5;  
3      a1 = arv_constroi('d', cria_arv_vazia(), cria_arv_vazia());  
4      a2 = arv_constroi('b', cria_arv_vazia(), a1);  
5      a3 = arv_constroi('e', cria_arv_vazia(), cria_arv_vazia());  
6      a4 = arv_constroi('f', cria_arv_vazia(), cria_arv_vazia());  
7      a5 = arv_constroi('c', a3, a4);  
8      a  = arv_constroi('a', a2, a5);  
9      return 0;  
10 }
```

Exemplo: Construção de árvore

Alternativamente, podemos criar a árvore com uma **única atribuição**, seguindo uma **estrutura “recursiva”**:

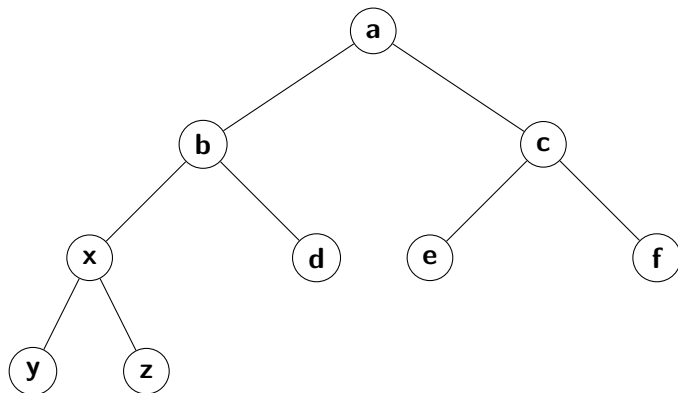
```
1  Arvore *a = arv_constroi ('a',  
2      arv_constroi('b',  
3          cria_arv_vazia(),  
4          arv_constroi('d', cria_arv_vazia(), cria_arv_vazia())  
5      ),  
6      arv_constroi('c',  
7          arv_constroi('e', cria_arv_vazia(), cria_arv_vazia()),  
8          arv_constroi('f', cria_arv_vazia(), cria_arv_vazia())  
9      )  
10 );
```

Construção de árvore

- Devemos notar que a definição de árvore, por ser recursiva, não faz distinção entre árvores e subárvores;
- A função `arv_constroi` pode ser usada para acrescentar (“enxertar”) uma subárvore em um ramo de uma árvore.

```
1  a->esq->esq = arv_constroi ('x',  
2  arv_constroi('y', cria_arv_vazia(), cria_arv_vazia()),  
3  arv_constroi('z', cria_arv_vazia(), cria_arv_vazia()))
```

Resultado

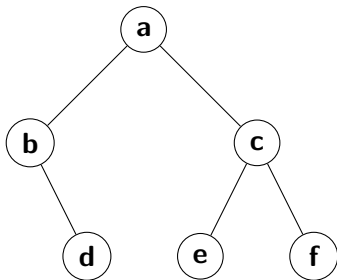


Exercícios propostos

Exercício 2 Para descrever árvores binárias, podemos usar a seguinte notação textual: a árvore vazia é representada por `<>`, e árvores não-vazias, por `< raiz esq dir >`. Com essa notação, a árvore abaixo é representada por:

`<a <b <> <d <> <> > ><c <e <><> ><f <> <> > > >`

Escreva uma função que recebe uma árvore de entrada e a imprime nessa notação.



Exercícios propostos

Exercício 3 Escreva uma função que retorna um valor booleano (um ou zero) que indica a ocorrência ou não de um dado caractere na árvore. Considere o seguinte protótipo para a sua função:

```
1 int arv_pertence (Arvore *a, char c);
```

onde `char c` é o caractere que deve ser procurado na árvore `a`.

Referências utilizadas na apresentação I